

NAME: V. Yugandhar

REG NO: 192472353

COURSE CODE: CSA0606

COURSE NAME: Design and Analysis of Algorithms

Q50. Debugging Insertion Sort Code (Wrong Direction in Shift Comparison)

1. Objective

To debug the given Insertion Sort implementation, identify the incorrect comparison condition, correct the shifting logic for ascending order, optimize the implementation for clarity, and analyze its time and space complexity.

2. Problem Identification

The given program uses `arr[j] < key` in the while loop. This condition shifts smaller elements to the right, which is appropriate for descending order, not ascending order. Therefore, larger elements are not shifted correctly and the sorted prefix is not maintained in ascending order.

3. Root Cause Analysis

Insertion Sort maintains a sorted prefix before index `i`. For ascending order, while scanning backward, an element should be shifted right only when it is **greater than the key**. The incorrect condition `arr[j] < key` moves smaller values and can place the key before larger values incorrectly.

4. Debugging Methodology

1. Read the source code and identify the shift condition.
2. Perform a dry run using a small array.
3. Track `key`, `j`, and `arr[j]`.
4. Observe that the comparison direction is reversed.
5. Change `arr[j] < key` to `arr[j] > key`.
6. Shift larger elements one position to the right.
7. Insert key at `arr[j+1]`.
8. Test the corrected algorithm with sample input.

5. Before vs After Code

Original Code:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] < key: # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Corrected Code:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Shift larger elements one position to the right
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Insert key into its correct position
```

```
arr[j + 1] = key

return arr
```

6. Step-by-Step Debugging Trace

Example: Array = [12, 5, 7, 5, 18, 9]

i	key	Action	Array after insertion
1	5	12 > 5 → shift 12 right	[5, 12, 7, 5, 18, 9]
2	7	12 > 7 → shift 12 right	[5, 7, 12, 5, 18, 9]
3	5	12 > 5, 7 > 5 → shift both	[5, 5, 7, 12, 18, 9]
4	18	No shift; 12 < 18	[5, 5, 7, 12, 18, 9]
5	9	18 > 9, 12 > 9 → shift both	[5, 5, 7, 9, 12, 18]

7. Why the Sorted Prefix Was Not Maintained

The sorted prefix must remain in ascending order after every iteration. With $\text{arr}[j] < \text{key}$, the algorithm shifts elements that are smaller than the key. This reverses the intended direction of insertion and may produce descending-style movement instead of placing the key after all smaller elements. Using $\text{arr}[j] > \text{key}$ guarantees that only larger elements move right, leaving the prefix sorted in ascending order.

8. Solution Implemented

The comparison condition was corrected from $\text{arr}[j] < \text{key}$ to $\text{arr}[j] > \text{key}$. The index j starts at $i-1$ and moves backward only while the current element is larger than key. After shifting, key is inserted at $\text{arr}[j+1]$, which is its correct position.

9. Optimization Techniques

- Clear variable names: key and j.
- Correct backward indexing with $j = i - 1$.
- Shift only elements larger than key.
- Insert key exactly at $j + 1$.
- Avoid unnecessary operations and keep the implementation concise.

10. Sample Input and Output

Input: [12, 5, 7, 5, 18, 9]

Output: [5, 5, 7, 9, 12, 18]

11. Complexity Analysis

Case	Time Complexity	Reason
Best Case	$O(n)$	Array is already sorted; one comparison per iteration.
Average Case	$O(n^2)$	Elements may need to be shifted multiple positions.
Worst Case	$O(n^2)$	Reverse-sorted input causes maximum shifts.
Space	$O(1)$	Only key and index variables are used.

12. Performance Evaluation

The corrected version performs correctly for ascending-order sorting. In the best case, when the array is already sorted, the algorithm runs in $O(n)$. In average and worst cases, shifting elements requires $O(n^2)$ time. The algorithm uses constant auxiliary space, $O(1)$.

13. Advantages

- Correct ascending-order output.
- Simple and easy to understand.
- In-place sorting with $O(1)$ extra space.

- Efficient for small or nearly sorted arrays.
- Maintains the sorted-prefix property after every iteration.

14. Limitations

- Average and worst-case time complexity is $O(n^2)$.
- Not ideal for large, randomly ordered datasets.
- More efficient algorithms such as Merge Sort or Heap Sort are preferable for large inputs.

15. Conclusion

The debugging process successfully corrected the wrong-direction comparison in the Insertion Sort algorithm. Replacing `arr[j] < key` with `arr[j] > key` ensures that larger elements shift right and the key is inserted at `j+1`. The corrected algorithm maintains a sorted prefix throughout execution, produces ascending order correctly, and uses $O(1)$ auxiliary space with $O(n)$ best-case and $O(n^2)$ average/worst-case time complexity.